Problem 12: Predictive Maintenance and Remaining-Useful-Life Estimation for Industrial IoT
Domain: Industrial AI | Time-Series | Anomaly Detection | Survival/Prognostics
Difficulty: Hard
Status: NEW (added to reach 15)

Problem

Unplanned machine failures cause costly downtime, while fixed-schedule maintenance wastes parts and labour. Sensor data is noisy, multivariate, drifting and has very few failure examples.

Objective

Build a system that ingests multi-sensor streams, detects early degradation, estimates remaining useful life (RUL) with uncertainty, and recommends maintenance timing that minimizes cost.

Users

Plant maintenance engineers, reliability teams.

Core Requirements

- Ingest multivariate sensor time series (streaming simulation).
- Preprocess: denoise, normalize per operating condition, handle missing sensors.
- Anomaly/health-index detection.
- RUL regression or survival model.
- Prediction intervals for RUL.
- Failure-mode or fault-type classification where labels exist.
- Explain which sensors drive the degradation signal.
- Maintenance recommendation using a cost model (failure cost vs early-replacement cost).
- Alerting with tunable thresholds.
- Dashboard: health trend, RUL, alerts, per-asset view.

Advanced Challenges

- Engine-wise (unit-wise) splits to avoid leakage.
- Multiple operating regimes and domain shift between units/machines.
- Label scarcity: semi-supervised or self-supervised pretraining.
- Asymmetric loss (late predictions cost more than early ones).
- Edge deployment: small model running on a Raspberry Pi/Jetson with latency measurement.
- Drift detection and model retraining trigger.

Suggested Datasets

NASA C-MAPSS (turbofan), FEMTO/PRONOSTIA and NASA IMS bearing datasets, CWRU bearing faults, SKAB (anomaly benchmark).

Suggested Tech

Python, PyTorch (LSTM/TCN/Transformer), scikit-learn, lifelines/scikit-survival, tsfresh, SHAP, MLflow, FastAPI, Streamlit; ONNX for edge.

Reference Architecture

Sensor simulator → stream processor (windowing) → feature/health-index module → RUL model + uncertainty → cost-based decision engine → alert service → dashboard; MLflow tracking + drift monitor.

Evaluation

RMSE and the C-MAPSS asymmetric score for RUL, interval coverage, anomaly detection precision/recall and lead time before failure, maintenance cost saved vs run-to-failure and fixed-schedule baselines, edge inference latency.

Pitfalls

Random window split mixing the same engine in train and test; evaluating only at end-of-life; ignoring operating conditions.

Suggested Timeline (48h)
0-8h data + baselines; 8-20h RUL models; 20-30h uncertainty + explainability; 30-40h cost-based maintenance + drift; 40-48h dashboard/edge demo.

Standard Judging Rubric (applies to every problem)

Problem understanding	10%
Technical depth	20%
Data engineering (quality, leakage prevention, reproducibility)	10%
Model performance (metrics, baselines, honest evaluation)	15%
System architecture	15%
Working prototype	10%
Explainability and responsible AI	10%
Innovation and scalability	5%
Presentation and demo	5%

Mandatory Submission Checklist

- □ GitHub repository with setup instructions
- □ Working prototype or recorded demo
- □ Documented system architecture
- □ Dataset sources and preprocessing methodology
- □ Evaluation results with baseline comparisons
- □ Limitations and failure cases
- □ One technically challenging feature demonstrated
- □ Short presentation: problem, approach, results, future work